is, the number of flagged transactions after which the session is blocked by the adaptive defense denoted by $\mathcal{D}_T$. We want to determine the probability that the session is not blocked by this adaptive defense. This is equivalent to the probability that strictly less than T out of the L transactions are flagged. If we let K_L denote the random variable representing the number of flagged transactions for a session of length L, then K_L follows a Binomial distribution with parameters L and $p_{\mathcal{D}}$ (using independence between the transactions). By averaging over the session length distribution, the SCR can therefore be computed as

$$\begin{aligned}\mathrm{SCR}(\mathcal{D}_T) &= \mathbb{E}\left[\mathbb{E}[\mathbb{P}(K_L < T) \mid L]\right] \\ &= \mathbb{E}\left[\sum_{k=0}^{T-1} \binom{L}{k} p_{\mathcal{D}}^k (1-p_{\mathcal{D}})^{L-k}\right]. \end{aligned} \tag{9}$$

The AFR and SCR computed in this way for adaptive defenses with varying threshold T based on each of the defenses in levels C1, C2, C3, and D (averaged over the LLMs) is shown in Figure 6 (the same plot but for `BasicUser` instead of `BorderlineUser` is shown in Figure S.6). Adaptive defenses with a low block threshold are able to significantly increase security over non-adaptive defenses that do not block entire sessions but only individual transactions. For instance, using a block threshold of 3 with the combined defense is able to block 75% of the attacks that were not caught by any of the non-adaptive attacks.

F. Additional Results: Defense-in-Depth and Adaptive Defenses

In this section, we provide additional results to the analysis in Section 4.2 of the main paper.

F.1. Defense-in-Depth

First, in Figure S.4 we provide a more detailed version of Figure 5 showing the same plot separated by LLM and setup.

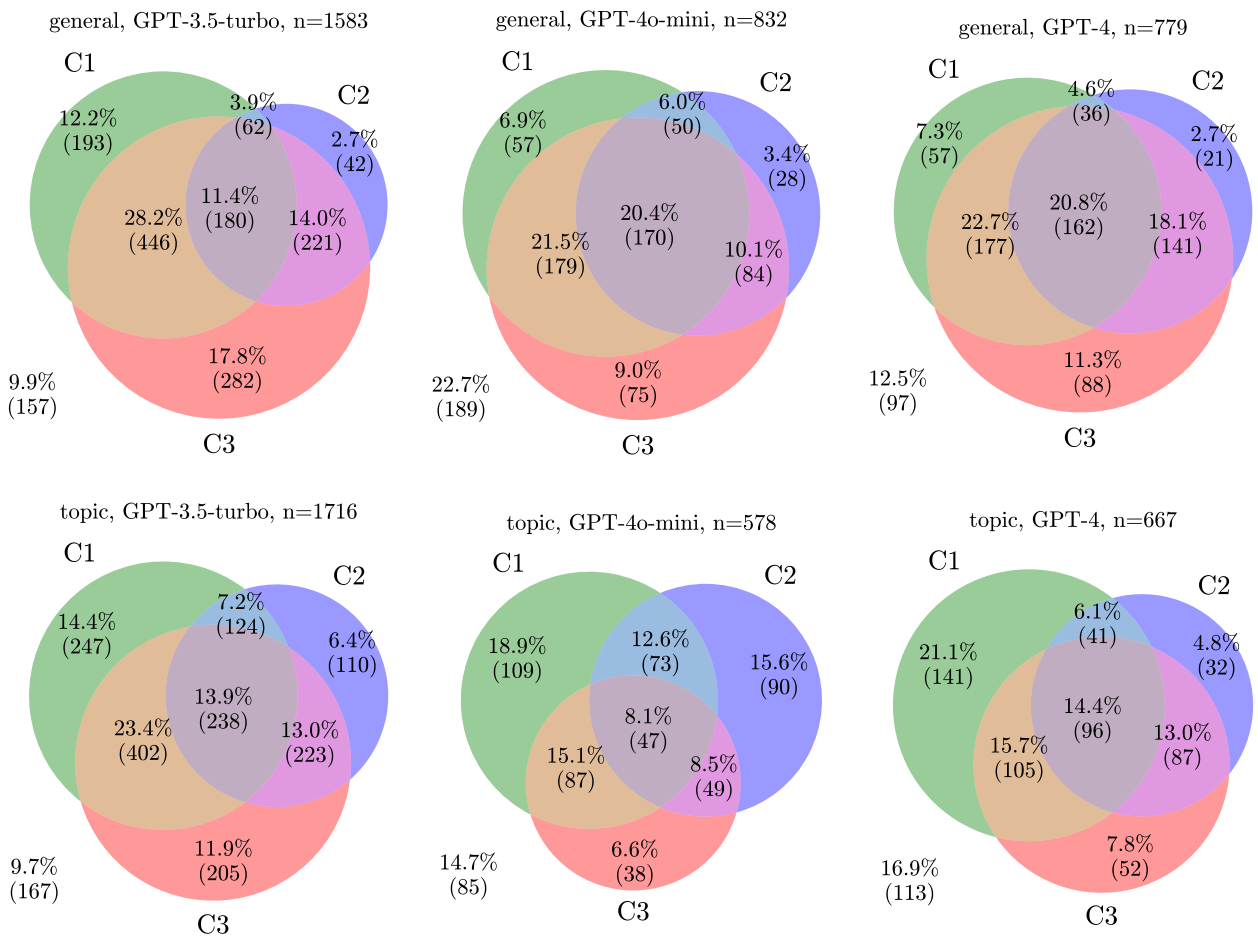

Figure S.4: *Detailed version of Figure 5 that shows that also after separating by setup and LLM, a defense-in-depth strategy is effective in increasing security.* In all cases, a fraction of attacks are caught exclusively by one of the defenses C1, C2, and C3, so their combination outperforms all individual defenses.

Second, as discussed in Section 4.2 we can explicitly optimize the aggregation function $f : \{0,1\}^3 \to \{0,1\}$ such that is maximizes developer utility, see (9). We compute it by expanding the developer utility $V^\lambda(\mathcal{D}_f)$ for the aggregated defense $\mathcal{D}_f$ explicitly (see Proposition E.1). For this purpose, we use the data processing discussed in Appendix E.2. The estimated developer utility for the 'or'-, 'and'- and f^*-aggregation for different trade-off parameters λ are provided in Table S.10. The results demonstrate that it is indeed preferable to use an optimized aggregation function in order to achieve a better utility-security trade-off.

F.2. Adaptive Defenses

Given a pre-selected developer utility, that is, a trade-off parameter λ (see (1)), we can explicitly select the block threshold T to maximize the developer utility. For the combined defense D, this is shown in Figure S.5 (left). For example, for $\lambda = 0.4$, the optimal block threshold is 2.

Table S.10: *Developer utility for different λ and aggregations.* In the case where only security matters ($\lambda = 0$), the optimal defense is to block everything, which is achieved with $f \equiv 1$. Similarly, if we only care about utility ($\lambda = 1$), the optimal defense does not block anything, which is achieved for $f \equiv 0$. When there is an actual trade-off ($\lambda \in (0, 1)$), we see that optimizing over the developer utility can provide a strictly higher utility than using the basic 'or'- or 'and'-aggregations. Here, f^* is the optimal aggregation defined in (9) with $\mathcal{D}_1 = \text{C1}$, $\mathcal{D}_2 = \text{C2}$ and $\mathcal{D}_3 = \text{C3}$ where we do not distinguish between LLMs.

λ	$V^\lambda(\mathcal{D}_f)$ 'or'	'and'	f^*	$f^*(x) = \mathbb{1}(x \notin \mathcal{X})$
0	0.87	0.15	**1**	$\mathcal{X} = \{\}$
0.25	**0.81**	0.36	**0.81**	$\mathcal{X} = \{(0,0,0)\}$
0.5	0.75	0.57	**0.79**	$\mathcal{X} = \{(0,0,0), (1,0,0)\}$
0.75	0.68	0.78	**0.85**	$\mathcal{X} = \{(0,0,0), (1,0,0), (0,1,0), (0,1,1)\}$
1	0.62	0.99	**1**	$\mathcal{X} = \{0,1\}^3$

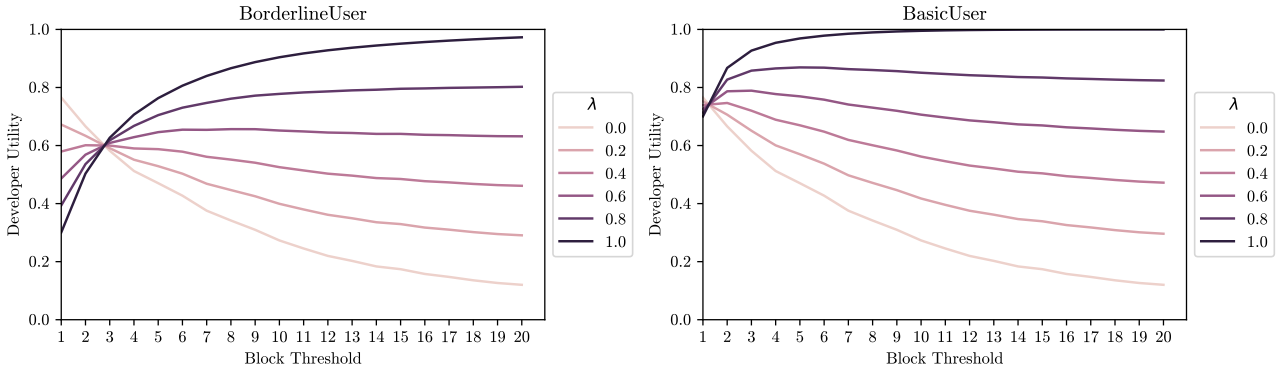

Figure S.5: *Optimizing the block threshold to maximize developer utility is possible.* Developer utility for different λ values and different block thresholds for `BorderlineUser` (left) and `BasicUser` (right).

For completeness, we include the results for the same analysis but with `BasicUser` instead of `BorderlineUser` in Figure S.5 (right) and Figure S.6. As expected the utility is much better, but the overall conclusion that one should optimize for security and utility remains the same.

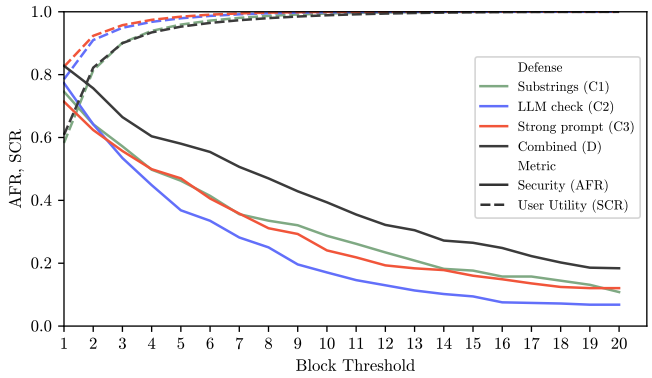

Figure S.6: *Adaptive defenses that block attempts past a block threshold enhance security.* Here, we use `BasicUser` instead of `BorderlineUser` to estimate utility. The same conclusions as for Figure 6 apply, even though the utility is substantially higher in this case.